MuleRadar

Catching money-mule rings while the money is still in the system

Challenge #5: Combating Digital Fraud and Financial Crime · Team: Abdussamad Ali, Abubakar Abdullahi, Umar Mohammed Bashir, Abdulbasit Idris Ali

MuleRadar watches the live transfer graph on a single GPU, recognises the **shapes that laundering leaves behind** (fan-in, fan-out, pass-through, scatter-gather and round-trip cycles), and raises a ring-level alert with plain-English reasons and a draft Suspicious Transaction Report. It holds the ring, not every unusual new customer.

AT A GLANCE (measured on the public IBM AML benchmark)

66.7%	90/95	99.6%	1.28M/s	15 ms
Laundering-transaction F1 best published: 64.8%	**Known laundering rings caught** test period	**Fewer innocent accounts flagged** vs threshold rule, equal recall	**Transfers scored per second** one A100 GPU	**Per batch of 2,048 transfers** IBM's CPU system: 148 ms

1. Problem & Opportunity

Fraud proceeds leave Nigerian banks through chains of mule accounts faster than today's controls can react.

- Nigerian financial institutions lost **₦52.26 billion to fraud in 2024**, three times the ₦17.67 billion of 2023 and 4.5× the 2020 level. Attempted fraud rose 338% in a year (NIBSS).
- The rails are fast and huge: NIBSS processed **11.2 billion e-payments worth ₦1.07 quadrillion in 2024**, about 355 transfers every second on average. Stolen money is split, passed across accounts and cashed out within hours.
- Most transaction monitoring still checks **one account at a time against fixed thresholds**, often in overnight batches. A mule ring is a network pattern: each account looks ordinary on its own, and the crime only shows in how money moves between them.
- Regulators are raising the cost of weak controls: seven of the largest Nigerian banks were fined a combined **US$10.7 million in 2024**, more than 60× the year before, including for AML failures, and CBN has introduced a fixed penalty regime for AML breaches.
- Blunt rules also **hurt financial inclusion**. Threshold rules put large numbers of legitimate customers under suspicion, and the newly banked (students, small traders, first-time digital users) are the ones whose accounts get frozen, while mule recruiters target exactly these groups.

The opportunity: detect whole rings, not single accounts, inside the window in which a hold can still stop the money, and do it precisely enough that banks freeze far fewer innocent customers.

2. Innovative Solution

MuleRadar is a streaming detection engine with an analyst copilot. Every incoming transfer passes through five stages:

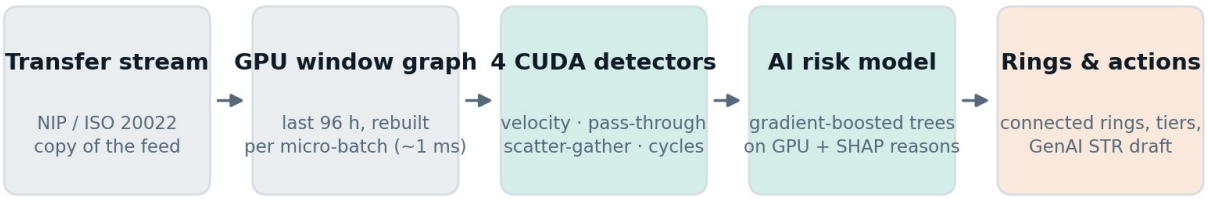


Figure 1. MuleRadar pipeline. Everything up to the risk model runs on one GPU.

1. **GPU sliding-window transaction graph.** Transfers arrive in time order. For each micro-batch the engine rebuilds the last 96 hours of the graph on the GPU (about 2 million transfers in roughly 1 ms) as time-sorted lists of each account's payments.
2. **Four custom CUDA detectors** compute, for every incoming transfer and using only the past, the laundering signals that published research identifies as decisive:
   - **Account velocity:** counts, value and number of distinct counterparties sent to and received from in 6-hour and 72-hour windows.
   - **Pass-through:** how much the sender received in the previous 48 hours, how much of it is being forwarded, and how soon after the last credit. Mules keep little and forward fast.
   - **Scatter-gather:** how many middlemen sit on time-ordered two-hop paths between the same sender and receiver.
   - **Temporal cycles:** whether this transfer closes a time-ordered round trip of 2 to 6 accounts, with money returning to where it started.
3. **AI risk scoring.** Gradient-boosted trees (XGBoost on the GPU) score each transfer. Alerted transfers are joined into rings, and every alert carries reason codes derived from the model's per-feature contributions (SHAP), for example "receiver collected from 10 different accounts in 72h".
4. **GenAI analyst copilot.** For each ring, a large language model drafts the Suspicious Transaction Report narrative for the NFIU from the ring's evidence only: accounts, transfers with IDs, timeline and reason codes. A validator rejects any draft that cites a transaction or account that is not in the evidence, and the LLM never makes the hold-or-clear decision. In production it calls an open-weight model on the bank's own GPU, so no customer data leaves the bank.
5. **Proportionate actions instead of blanket freezes:** monitor, then step-up verification (BVN selfie or in-app confirmation), then hold outbound transfers on ring members, then escalate with an STR.

Where AI, GenAI and emerging technology are used

Technology	Role in MuleRadar	Why it matters
GPU computing (CUDA)	Four purpose-built kernels for temporal laundering patterns; the transfer graph lives in GPU memory	Real-time network analysis at national payment volumes on one server
Machine learning	Gradient-boosted trees on 48 behavioural and network features, trained on the GPU	Precise scoring: 84.8% of alerted transfers are true laundering
Explainable AI (SHAP)	Per-alert reason codes in plain English	Analysts and regulators can see why an account was flagged
Generative AI (LLM)	Drafts the NFIU Suspicious Transaction Report from the ring's evidence, with a grounding check	Cuts analyst writing time; the human approves every report

Technology	Role in MuleRadar	Why it matters
Graph analytics	Alerts are linked into rings of connected accounts	Act on the whole network, not one account at a time

What is new: the detectors are not generic graph libraries. They are purpose-built CUDA kernels for time-ordered laundering patterns, verified to give exactly the same output as a CPU reference implementation, running in a streaming loop on one GPU.

3. Impact & Value

All numbers below were measured by us on the public IBM "Transactions for Anti-Money Laundering" benchmark (HI-Small: 5.08 million transfers, 515,000 accounts, 18 days), replayed as a stream, with **the same time-ordered 60/20/20 train, validation and test split used by the published papers**. The test set is the last 20% of the stream and was never seen during training.

Result	MuleRadar	Reference
Laundering-transaction F1	**66.7% ± 0.3** (3 seeds)	IBM Graph Feature Preprocessor + XGBoost (2024): 63.2–64.8%; PNA graph neural network: 56.8%; plain XGBoost: 19.8%
Precision of alerts	**84.8%** (recall 54.9%)	
Known laundering rings flagged	**90 of 95** (95%)	All cycle, scatter-gather, gather-scatter and stack rings caught
Innocent accounts flagged per 10,000	**7.6**	1,950 for a 24-hour outbound-amount rule at the same recall: **99.6% fewer** wrongful flags
Throughput on one A100 GPU	**1.28 million** transfers/s	15× the same algorithms on 8 CPU cores (23× at batch 2,048); IBM's system: about 14,000/s on 64 CPU threads
Latency per batch of 2,048 transfers	**15.2 ms**	IBM's system: 148 ms

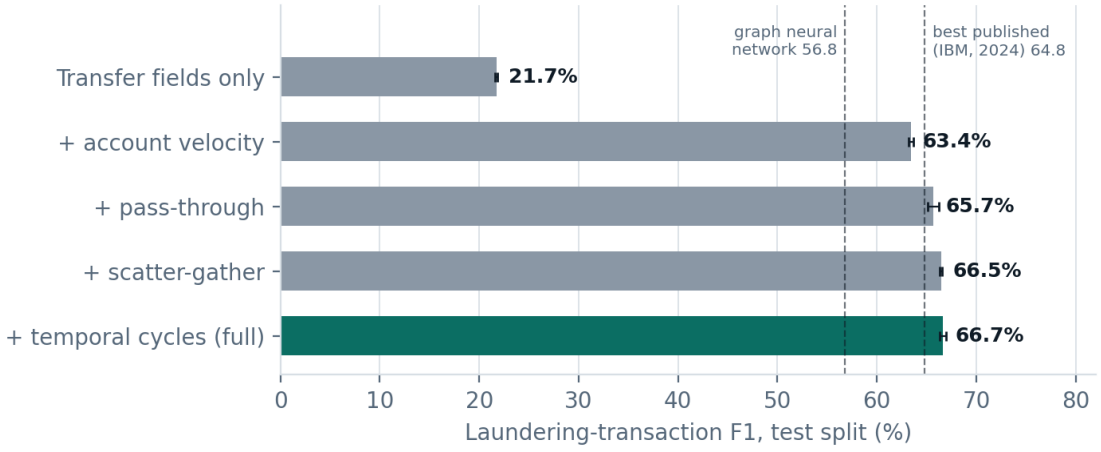


Figure 2. Each detector family adds accuracy (mean ± std over 3 seeds).

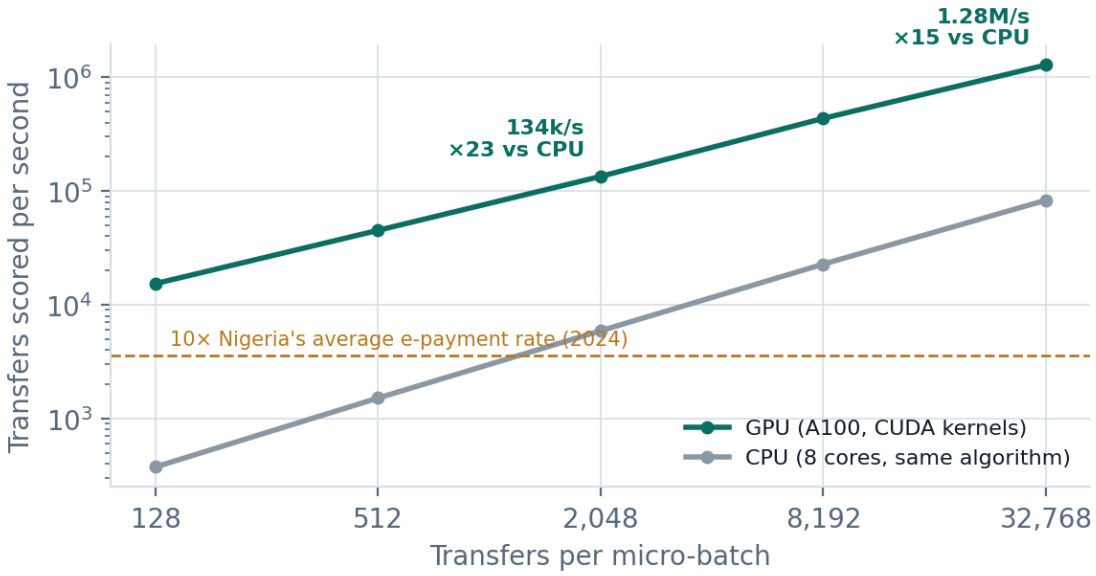


Figure 3. Same algorithm on GPU and CPU, end to end per micro-batch.

What this means for a Nigerian bank

- **Fewer wrongful freezes, more inclusion.** To catch the same half of the mule accounts, a typical amount rule would put 70,616 accounts under suspicion. MuleRadar flags 1,401, of which 81% are truly involved. For students and small traders, that is the difference between a working account and a frozen one.
- **Capacity to spare.** Nigeria's 2024 average is about 355 e-payments per second. One GPU sustains over a million per second, so a single server covers a large bank's peak load with wide headroom, and the same software runs on a cheaper GPU for microfinance banks and fintechs.
- **Analyst time.** Each alert arrives as a ring with reasons and a drafted STR narrative, instead of a list of isolated threshold hits.

Limits we state openly

The benchmark is synthetic, because real bank labels are confidential. In it, laundering is mostly generated as one payment format, which lets every model, ours included, alert on a ring's very first transfer; our measured "95% of ring value still stoppable at the first alert" is therefore an upper bound. On real data the first alert will come a few hops in. The network detectors are what carry detection once that shortcut is gone, and the ablation shows they lift F1 from 21.7% to 66.7%.

4. Target Users & Implementation

Who benefits. Fraud and AML operations teams and compliance officers at commercial banks first; then microfinance banks, payment service banks and fintechs, which carry the same CBN and NFIU obligations with much smaller teams. Customers benefit directly through fewer wrongful freezes and faster recovery of stolen funds.

How it deploys

- Reads a copy of the transfer feed (NIP / ISO 20022 messages or the core-banking transaction log). No change to core systems.
- Runs on **one on-premises GPU server**. Customer data never leaves the bank, and the copilot LLM is an open-weight model on the same hardware.

- Sends alerts to the bank's existing case-management tool through an API. Analysts approve every hold and every STR.

Rollout

1. **Back-test** on 3 to 6 months of the bank's historical transfers and confirmed fraud cases.
2. **Shadow mode:** live alerts go to analysts only, and precision and wrongful-flag rates are measured against the current system.
3. **Analyst-in-the-loop holds** on tier-3 and tier-4 rings.
4. **Automated step-up verification** for tier-2 accounts.

Feasibility today

Every result in Section 3 was produced by our working prototype: data pipeline, CUDA engine, CPU reference, model training, ring evaluation, STR drafting and the analyst console (Figure 4). A live mode replays the held-out transfer stream through the engine on the A100 and streams alerts and rings to the console in the browser, with a switch to run the same detectors on the CPU for comparison. It is built in Python with CuPy (custom CUDA C++ kernels), Numba, XGBoost, Polars and FastAPI, and each step is reproducible with one script. The STR module is wired for an on-premises LLM endpoint; in the prototype the drafts come from the evidence-grounded template, because our GPU cluster could not download model weights during the build.

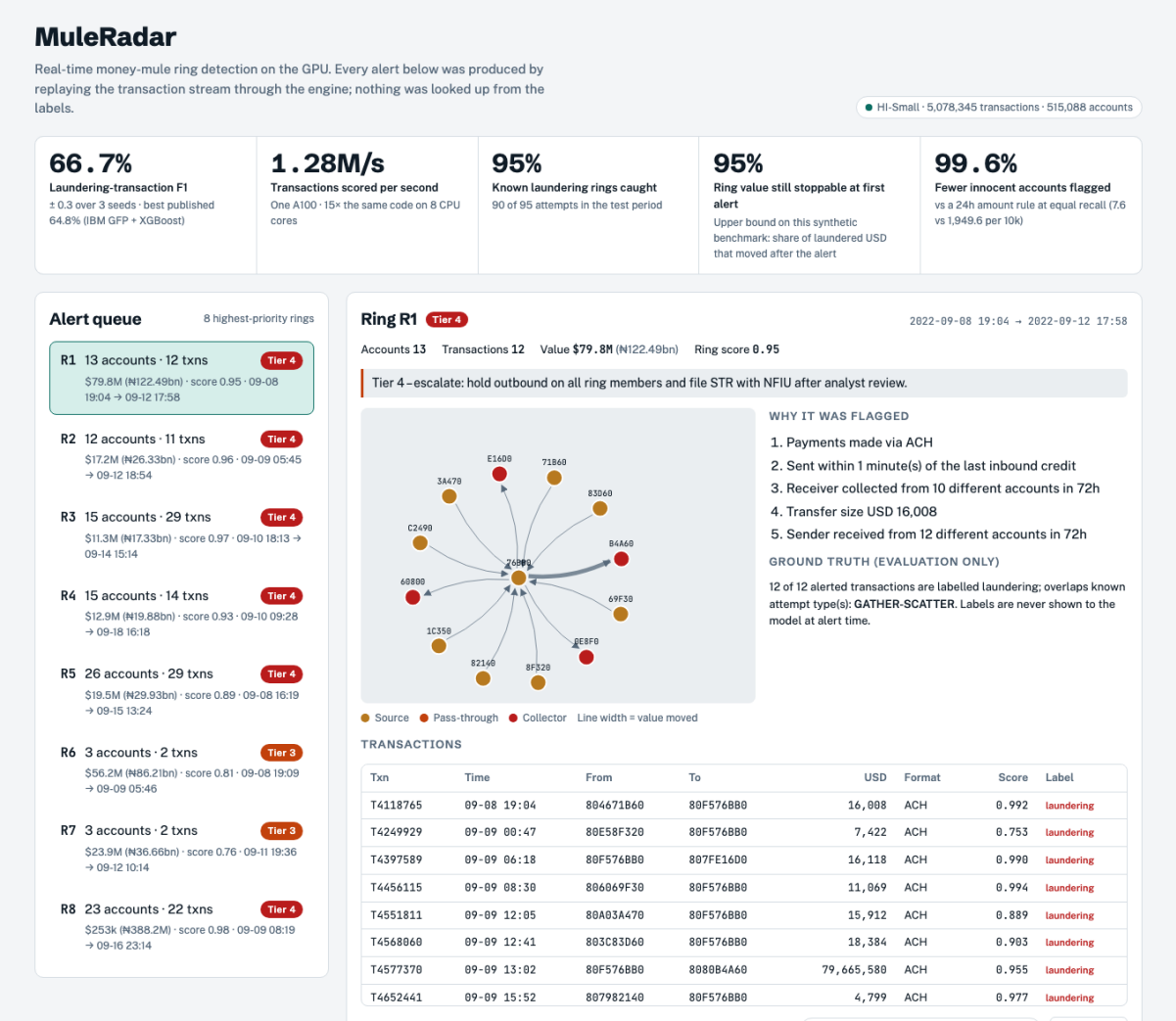


Figure 4. MuleRadar analyst console: ring queue, money-flow graph, reason codes and transfers.

Business model. An annual licence per institution, tiered by transaction volume, with a managed-appliance option for microfinance banks that have no GPU infrastructure.

5. Team & Expertise

Member	Role	Contribution to MuleRadar
Abdussamad Ali	CEO	Leads the venture and the GPU and AI build: problem framing, the CUDA detection engine and model evaluation. Owns bank partnerships and go-to-market.
Abubakar Abdullahi	CTO	Owns the technical architecture and deployment path: streaming ingestion from NIP and core-banking feeds, the on-premises GPU server, security, and integration with case-management tools.
Umar Mohammed Bashir	Product Manager	Turns detections into analyst workflow: tiered actions, the STR drafting flow for the NFIU, CBN and NFIU compliance requirements, and the pilot rollout plan.
Abdulbasit Idris Ali	UI Designer	Designs the analyst console: alert queue, ring visualisation, plain-English reason codes and the review-and-approve STR screen.

Together the team covers what a bank needs in order to adopt MuleRadar: a detection engine that works at national scale (CEO and CTO), a workflow that fits compliance obligations (Product), and an interface analysts can act on in seconds (Design).

Sources

- NIBSS 2024 fraud report (via 21st Century Chronicle)
- NIBSS 2024 e-payment volumes (TechCabal Insights)
- Regulatory fines on Nigerian banks, 2024 (BusinessFront)
- Blanuša et al., Graph Feature Preprocessor, ICAIF 2024
- Egressy et al., Provably Powerful GNNs for Directed Multigraphs, AAAI 2024
- Altman et al., Realistic Synthetic Financial Transactions for Anti-Money Laundering Models, NeurIPS 2023 (IBM AML dataset)